

fsb: Design Specification

Version 1.0

October 2, 2026

Describes the code at commit [2c4738a](#)

Abstract

This document describes how **fsb**, the filesystem browser, is built and why it was built that way: its structure, the mechanism behind each guarantee in the functional specification, the alternatives that were considered and rejected, how the design is tested, and what it does not do. It is written for people who maintain the program, review it, or test it adversarially. It changes when the construction changes; the functional specification does not. Requirement identifiers such as RUL-6 refer to the functional specification. Section [11](#) states exactly where and with what tools this version was tested; Section [13](#) states, just as plainly, what was not.

Contents

1	Principles	3
1.1	Technology	3
2	Architecture	4
2.1	Components	4
2.2	Structure	4
2.3	Life of a request	4
2.4	Startup	5
2.5	The port, and background mode	5
3	The Guard	6
3.1	Opening a path	6
3.2	Aliases	6
3.3	Roots	7
3.4	Permission errors inside denied places	7
3.5	Listing	7
3.6	Search	7
3.7	Cloud-only files	8
3.8	Other files and endpoints	8
3.9	Hard links	8
4	The Rule Matcher	10
4.1	Matching	10
4.2	Compiling a rule	10
4.3	Name equality	10

4.4	Speed	10
4.5	Loading a rule file	11
4.6	The built-in rules	11
5	The HTTP Layer	11
5.1	Middleware	11
5.2	The session and the prefix	12
5.3	Headers and framing	12
5.4	Streaming	12
5.5	Errors	12
5.6	Streams end explicitly	12
5.7	Timeouts and shutdown	12
6	Content-Typed Endpoints	13
6.1	Images	13
6.2	PDFs	13
6.3	Quick Look pictures	13
6.4	Archives	14
6.5	Rendered Markdown	15
7	The Front End	15
7.1	Structure	15
7.2	Decisions	15
7.3	The preview pane as a state machine	17
7.4	Hover peek	17
8	Verification	17
8.1	Layers	18
8.2	Continuous integration	18
8.3	What testing does not establish	18
9	Decisions and Alternatives	19
10	Limitations, Risks and History	21
10.1	Risks	21
10.2	Known limitations	22
10.3	History of defects found by the design's own checks	22
10.4	Milestones	25
11	Tested Environment	25
11.1	Hardware	25
11.2	Operating system and browsers	26
11.3	Go toolchain and modules	26
11.4	Front end: Node and npm packages	26
11.5	Build and documentation tools	27
11.6	Regenerating this section	27
12	Requirement Traceability	27
12.1	Requirements with no automated check	34

13 What Was Not Tested	35
13.1 Platforms and browsers never run	35
13.2 Scenarios inside the test suite that did not run this time	35
13.3 Process and infrastructure	36
13.4 What this does not mean	37

1 Principles

The security of `fsb` rests on a few principles that the rest of the design applies. Each was, at some point, the lesson of a defect (Section 10).

One chokepoint. Every read of the user’s file system goes through one package, the *guard*. No package that serves HTTP may touch the file system, and a test fails the build if one imports `os`, `io/fs`, `syscall` or `path/filepath`. A property of the guard therefore holds for every endpoint at once.

Fail closed. When a decision cannot be made, the answer is refusal: a malformed rule file stops the program from starting; a file with several names is refused until its names are known; a path that cannot be resolved is not served.

Over-match to deny, decide by identity to allow. Comparing names can be wrong in either direction. A deny check may safely match too much (it only blocks more); an allow check must not, and so decides by the identity of the file or folder (device and inode), never by the spelling of its name.

Name equality belongs to the volume. Whether two names are the same file is a fact about the file system, not about the program. The program compares names the way the volume does, and tests that claim against the operating system rather than against its own model.

Decide from bytes, not names. What a file *is* (an image, a PDF, an archive) is decided from its contents. A name is at most a hint.

Read nothing that is not needed. Refusal is decided before any byte is read; a cloud-only file is never read implicitly, because reading it downloads it.

Each layer assumes the one before it failed. Rendered Markdown, for example, is escaped, then sanitized, then shown in a frame that cannot run anything of its own or reach the network.

Verify against the environment. The tests that matter most let the operating system decide the facts (Section 8).

1.1 Technology

The back end is Go, using the standard library’s HTTP server, with one extra dependency for Unicode (golang.org/x/text, for normalization and case folding) and one for extended attributes (golang.org/x/sys). The front end is Svelte 5 with TypeScript, built with Vite, and embedded in the binary with `go:embed`; the compiled output is committed so that building the binary needs no Node. The front end has three runtime dependencies that ship in the output: `marked` (Markdown),

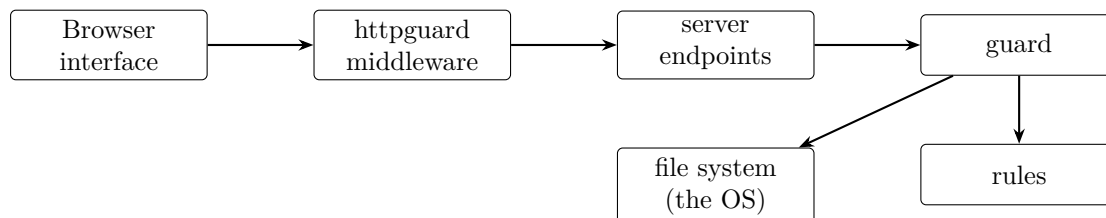
dompurify and highlight.js; Svelte is compiled away. The result is a single binary of about 11 MB.

2 Architecture

2.1 Components

Component	Responsibility
cmd/fsb	Command line, startup, the launch URL, opening the browser, shutdown; the remembered port, background mode and finding a running fsb (<code>config.go</code> , <code>background.go</code> , <code>process.go</code>).
internal/rules	The rule dialect: parsing, matching, name folding, the built-in rules. Knows nothing of files.
internal/guard	The chokepoint: opening, listing, searching, reading heads, metadata, sniffed endpoints, archive listing, the hard-link index. The only code that touches the file system on behalf of a request.
internal/httpguard	The middleware that makes a local web server safe: host and origin checks, the session, the prefix, security headers. Knows nothing of files.
internal/quicklook	Runs Apple's <code>qlmanage</code> to draw a Quick Look picture of a private copy that the guard made. The only code that runs another program on a file. Knows nothing of rules.
internal/server	The endpoints. Turns guard results into HTTP, and guard errors into status codes.
web	The embedded front end, and the fixed page that renders Markdown.
e2e	Browser tests, and their drivers for Chrome and Safari.
specs	These documents.

2.2 Structure



Dependencies point one way: **server** uses **guard** and **httpguard**; **server** also uses **quicklook**, which draws only the copy that **guard.CopyForQuickLook** made; **guard** uses **rules**; **rules**, **httpguard** and **quicklook** use nothing of the program's own. **cmd/fsb** assembles them.

2.3 Life of a request

1. The HTTP server accepts the connection on the loopback address.
2. The access log records the method, path and status, never the query (which is where the launch token travels).
3. The middleware sets the security headers, then checks, in order: the **Host** header; the method; the **Origin**, **Referer** and **Sec-Fetch-Site** headers; that the path is under the random prefix (which it strips); the launch-token exchange; and the session cookie. A failure of any check is the same 403 (SEC-2 to SEC-8).

4. The endpoint reads its parameters and calls one guard operation.
5. The guard canonicalizes the path, checks it against the deny rules before and after opening, opens the file, verifies that what was opened is what was checked, and decides the file's kind from its bytes where needed.
6. The endpoint turns the result into a response, and every guard error into a status code in one function (**fail**), so that denied, missing and outside-the-roots paths cannot be told apart by accident.

2.4 Startup

`cmd/fsb` resolves the home folder to its real path. `-init`, `-stop` and `-status` act and exit here, and `-background` starts its child here (which runs these same steps from the start). It then loads the two rule files (or their built-in defaults); `-show-deny` and `-show-ignore` print them and exit here, so they show exactly what would be enforced. It then builds the guard from the roots (served once each, compared by identity) and rules, names the credential locations the guard must also protect, starts the hard-link index building in the background, binds the loopback port (`choosePort`, Section 2.5), prints the launch URL, and only then serves. A failure at any step before serving stops the program with a message.

2.5 The port, and background mode

Browsers keep local storage per origin, and the origin includes the port, so a port chosen afresh at each start would reset every preference (CFG-4). `choosePort` therefore reuses the port in `~/.config/fsb/lastport`, and writes it after a start that did not use `-port` (best effort, like the front end's own writes to local storage). The random port was never a security control: the boundary is the 256-bit launch token, the `Host` and `Origin` checks, and the random path prefix, which still changes at every start. `-port` is deliberately not remembered, so that a one-off choice made to avoid a conflict heals back to the usual port, and its preferences, once the conflict is gone. If the remembered port is taken, `fsb` fails rather than silently moving (which would look as if it had lost the preferences); `portInUseMessage` asks `lsof` and `ps` which program holds it so that the message can name it.

`-background` re-executes the program with `FSB_BACKGROUND_CHILD=1` in a new session (`setsid`), so the child has no controlling terminal. The child's standard output and error are a writer (`handoff`) that goes to a pipe on file descriptor 3 until the child is serving, then to `~/.config/fsb/background.log`. The parent copies the pipe to its own output until a fixed ready marker arrives (success: it releases the child and exits) or the pipe closes without one (the child failed and has printed why; nothing is left running). The child records its process number in `~/.config/fsb/pid` and removes it on shutdown if it still names itself.

`findRunning` locates a running `fsb` for `-stop`, `-status` and the refusal of a second `-background`: first the process in the PID file, then the process listening on the remembered port. Either is accepted only if `ps` belongs to the same user and its executable file (from `/proc` or `lsof`, not the name `ps` shows, which on macOS is the process's own `argv[0]`) is named `fsb` or as the running binary is, and a process from the PID file must also have the start time recorded there. So a stale PID file whose number was reused, or another program on the port that calls itself `fsb`, is never signaled. `fsb`'s own files are opened without following symbolic links and set to mode 0600.

3 The Guard

3.1 Opening a path

Every operation begins by opening the path, in this order. The order is the design.

1. **Canonicalize.** The path must be non-empty, contain no NUL, and be absolute. It is cleaned lexically (`.`, `..` and repeated slashes removed) and reduced to one spelling of each alias (Section 3.2). URL-decoding has already happened, exactly once, in the HTTP layer; the guard never decodes, so a literal `%2e%2e` is just a name.
2. **Lexical deny check.** The cleaned path is matched against the deny rules *before the file system is touched*. The type of the final component is not yet known, so it is treated as a file here; a rule that applies only to folders is decided in step 7. (Assuming a folder would refuse a plain file that shares its name with a `dir/` rule although listings show it.)
3. **Open without blocking.** The path is opened read-only and non-blocking, so that a pipe or a device cannot hang the server. A permission error is examined (Section 3.4) before it is reported.
4. **Reject special files.** On the descriptor, anything but a regular file or a folder is refused. Only then is blocking restored. Some special files never reach a descriptor to reject this way: opening a UNIX domain socket's path fails at the syscall itself (`EOPNOTSUPP` on macOS, historically `ENXIO` on Linux), so those two `errno`s are mapped to the same refusal as every other special file, rather than left to fall through as an unrecognized error (Finding 17).
5. **Resolve and confirm.** The real path is resolved (symbolic links followed), and the guard confirms that the file at that real path is the *same file* as the open descriptor (device and inode). If not, the path changed between the two steps and the request is refused as a race.
6. **Roots and deny on the real location.** The real path must lie inside a root (by identity, Section 3.3), and must not match a deny rule.
7. **Precise lexical check.** With the type now known, the requested path is matched again. A path is thus refused if *either* its spelling or its real location is denied.
8. **Other names.** A regular file with several names is checked against the hard-link index (Section 3.9).

Everything that later reads bytes does so through the descriptor this returns, so the decision and the read concern the same file. Extended attributes, for example, are read from that descriptor.

3.2 Aliases

macOS keeps its user data on a separate volume whose folders appear at ordinary paths through *firmlinks*: `/Users` and `/System/Volumes/Data/Users` are one directory. Firmlinks are not symbolic links, so resolving links does not reduce them, and a rule for `/Users/me/.ssh` would not match the longer spelling. Every path is therefore reduced to its ordinary form before any comparison with a root or a rule.

The reduction compares path components one at a time, with the same name equality as the rules (Section 4.3), and rebuilds the path from the firmlink's own spelling. An earlier version sliced the

path to the byte length of the prefix, which cut a folded spelling (*/Slong-system*) in the middle of a character and left it unreduced. The firmlink table is read from `/usr/share/firmlinks`, with a built-in copy as fallback.

3.3 Roots

Each root is resolved to its real path at startup, and its identity (the file information of the folder) is kept. Whether a path lies inside a root is then decided by taking the leading components of the path that correspond to the root and comparing the *identity* of that folder with the root's. Comparing folded text would be right on a case-insensitive volume but would admit a different sibling on a case-sensitive one (`Public` for a root of `public`). This is an allow decision, so it must not over-match.

3.4 Permission errors inside denied places

An unreadable file inside a denied folder must look exactly like a missing one, or its existence leaks. When opening fails with a permission error, the guard resolves the longest prefix of the path that can be resolved (an unsearchable folder stops the resolution of what is inside it, not of the folder itself), rebuilds the full real path from it, and applies the deny rules to that. Only if nothing denies it is the permission error reported.

3.5 Listing

A listing opens the folder as above and receives its real location. Each child is then judged by *that* location and not by the symbolic link or alias the folder was reached through, in one function used by both listing and search (`entry`), so that search cannot show what listing would not. For a child:

- a symbolic link is resolved; a broken one is marked; one whose target is denied or outside the roots is omitted; the target's size, date and type are shown;
- the child's path is matched against the deny rules and, separately, the hide rules, each ignoring the ancestors above the folder itself (so entering a hidden folder on purpose shows its contents);
- a regular file with several names is checked against the hard-link index.

Entries are handed out in batches as the directory is read, which is what lets the interface paint before a huge folder has been read.

3.6 Search

Search is a breadth-first walk, so shallow matches come first. Every folder is opened through the same operation, every child goes through `entry`, denied and hidden folders are never entered, and symbolic links to folders are never followed (which also rules out cycles). It stops at a result limit or an entry limit and reports that a limit stopped it. Names are compared after the folding of Section 4.3, or, for *Match case*, after normalization only.

Three details keep the walk honest. The entry limit counts every entry *examined*, including those a rule then rejects, because it bounds work; the count reported back is of visible entries only, so it says nothing about how many were excluded. Folders are queued by path, so before one is entered its real location, as the guard's own open reports it, is compared with the name it was queued under; if

a symbolic link has taken its place (or one above it) in the meantime, the children would be judged under a name that is not where they are, so it is not entered (a test swaps a queued folder for a link after the walk has queued it). Finally, the search returns a summary beside its results: a folder that is there and allowed but could not be opened, one that failed partway through being read, and one skipped as just described count as not searched, and the done line of the stream carries `incomplete` so the interface says so. A folder that is denied or has vanished is not counted: a denied folder must look exactly like a missing one, and neither is a loss. Cloud-only folders are skipped by design and do not count either, so completeness means completeness of the searchable, allowed content.

3.7 Cloud-only files

On macOS a file that exists only in the cloud carries a flag in its file information. The guard checks it before any read and reports the file as not downloaded; the head, the metadata, the previews and the archive listing all refuse to read it. Reading such a file would make the system download it, which `fsb` must never do implicitly. An explicit download by the user is the one path that reads it. The hard-link index walk (Section 3.9) and the copy of a package for Quick Look do not enter a cloud-only folder either: the walk skips it (its contents are not on this disk, so they hold no other name of a file that is), and a package with a cloud-only part is refused.

A *folder* can carry the same flag: the newer File Provider domains (iCloud Drive, OneDrive and similar) can make a whole folder a placeholder, not only the files inside it, and reading its contents is exactly as capable of triggering a download as reading a dataless file's bytes is. Listing and search each check the flag on a directory before opening it for reading, refusing it the same way as a dataless file, rather than only on the files this directory would contain. This was missing for a released cycle: nothing had ever put a real File Provider placeholder folder in front of the guard, only a synthetic flag on individual files, so listing one fell through to the directory read and surfaced whatever the provider's virtual file system returned for it, unrecognized, as an internal error (Finding 19).

3.8 Other files and endpoints

Head reads at most a stated number of bytes from the descriptor, and returns text only if it is valid UTF-8 without control characters; binary contents never leave the guard.

Metadata describes the entry, and reads extended attributes through the descriptor, capped in number and size.

Sniffed endpoints (image, PDF, archive) share one routine that opens the file, reads its first bytes, decides the kind from them, and rewinds; they differ only in the test applied (Section 6).

Quick Look copy (`CopyForQuickLook`) opens the document as above and copies bytes from the verified descriptor into a private folder; in a package it opens every part through the guard by full path, so each part is judged exactly as a direct request for it would be (Section 6.3). It writes only into the temporary folder it is given, never into the roots.

3.9 Hard links

Rules name paths, and a hard link is another name for the same file, so a hard link to a denied file is invisible to them. The guard therefore judges a regular file with more than one name by its identity: if any of its names is denied, all of them are.

The index. Finding the other names of a file means visiting every file under the roots, because the operating system offers no way to ask. A walk therefore builds an index from identity (device and inode) to what it saw of the file: its names as real paths, the link count it saw, and whether any name is denied. It contains only regular files with more than one name (a few dozen on a typical home folder). The walk also covers the *protected locations* named by the program at startup (the credential folders of every home), so that a link from a served folder to a credential file outside the roots is found. A name is identified by the folder holding it (by identity) and its last component folded as the file system compares names (Section 4.3); two paths that spell one name (a firmlink, a folder walked twice, `a.txt` and `A.TXT`) therefore count once. On a case-sensitive volume two names that differ only in case also count once, which undercounts and so refuses: the safe direction.

The walk is in the background. On a home folder of 700,000 files it takes about 18 seconds, which must not be spent inside a request. It is started at startup, and nothing is served as a multi-name file until it has finished. Walks are spaced by at least twice the time the last one took (and never closer than a fixed cooldown), and are bounded by an entry count and a time. A cloud-only folder is not entered.

The policy: verify all the names. A file with several names is served only when the index accounts for every one of them, now. The decision (`judgeLinked`) requires that the walk found as many distinct names as the file has links, that none is denied, and that each recorded name is still a real path to this very file when it is used: it is looked at again, resolved, and must still reach the same identity with no symbolic link along it. Nothing about the file itself is trusted to reveal a change, because renaming or moving a folder renames the files in it without touching them (an earlier design that trusted a file whose change time was old enough was broken exactly this way by a second review, and was replaced). Anything else is refused, never guessed: the index is not ready, a name lies where the walk does not look (a pnpm store outside the roots) or in a folder it could not read, the walk stopped at its limits, or the names changed. This was once the other way round for a file the walk could not see (it was allowed after a second walk failed to find it); a review showed that fail-open policy let a hard link to a denied file through, and the decision is now fail closed throughout.

Retry. A pair made while a walk runs can be recorded with one name, and one more walk settles it. That walk is owed once per file and link count, and is spent only when it really starts: a first look during the cooldown, or while another walk is running, refuses the file without using it up (an earlier version marked it spent before checking, so the file stayed refused after the cooldown). If the extra walk still finds fewer names than links, the file is refused without walking again until something changes.

Listings and searches reuse a verdict for one second. A folder holding many names of one file would cost one check per name, each a look at every recorded name, so a listing or a search remembers the verdict for a file (keyed by identity and link count, so a name added meanwhile is noticed) — but only for a short, stated time. After one second (`linkMemoTTL`) the names are checked again, because a rename that leaves the count unchanged, one of two names moved onto a denied name, would otherwise stay invisible for as long as a slow consumer or a long search ran. This is a memo reuse interval, not a guarantee that every row on the screen reflects the file system now: entries already sent are not retracted, and the check made when a file is *opened* never uses the memo. The remaining staleness affects a name or a size shown, not access to contents.

What is not covered. A denied name that lies outside every root and every protected location is not found; the file is then refused while the file has links the walk cannot account for, rather than served.

4 The Rule Matcher

4.1 Matching

A rule set is an ordered list of compiled rules. For a path, the matcher evaluates each prefix of the path in turn (the folder itself, its parent, and so on down to the path), and the set *excludes* the path if any prefix is excluded: an excluded folder excludes everything in it, and no later rule (not even a negation) can bring it back, as in git. Within one prefix the last matching rule wins, which is what makes negation in the ignore file work. The deny file has no negation, so it is a plain union of rules.

Matching a path *below* a folder ignores the prefixes at or above that folder. It is used for the hide rules while listing, so that entering a hidden folder on purpose shows what is inside it.

4.2 Compiling a rule

A rule is translated to a regular expression, case-insensitive and with the dot matching a newline (so that a name containing a newline cannot defeat ******), ending in `\s*$`, so that a name with trailing white space still matches. The translation follows the gitignore glob: ***** and **?** do not cross a separator; ****** matches folders; a class `[...]` never matches a separator (a negated class excludes it explicitly, and a `/` inside a class is an error); a trailing `/` makes the rule apply to folders only. A pattern is anchored to the home folder if it begins with `~/`, is absolute if it begins with `/`, is relative to the home folder if it contains a `/` elsewhere, and matches at any depth otherwise or after a leading `**/`.

4.3 Name equality

Both the pattern and the path are folded before they are compared (**Fold**): Unicode normalization (NFC), then *full* case folding, then NFC again. The rules must match names as the volume compares them, and on macOS the case-insensitive volume uses full folding, under which the sharp s equals **ss**, the long s equals **s**, the Kelvin sign equals **k**, and the ligatures **ff**, **fi**, **fl**, **ffi**, **ffl** and **st** equal their spelled-out forms. Simple case folding, which was used at first, treated these as different names, so `~/fsh` opened `~/ssh` while the rule for `.ssh` did not match. The set of aliases was *measured* on the volume, by creating every one- and two-letter name and probing every character in the Unicode range, and is a test. The client-side filter uses the equivalent lower-, upper-, lower-casing, and the tests check that the two agree on the measured aliases.

Names are compared this way so that a deny rule over-matches, which is safe. Where the result is an *allow* (whether a path is inside a root), identity is used instead (Section 3.3).

4.4 Speed

Listing a folder evaluates the rules once per entry. A rule set of about thirty rules took about 29 μ s per entry, which would add three seconds to a folder of 100,000 entries, so two optimizations were made, both of which must give the same answer as the plain loop:

- A rule whose pattern contains no slash can match only the last component of a path, so it is tested against that component alone (short and anchored) instead of against the whole path at every level.
- For a set without negation, all rules of each kind are combined into one expression, asked first; only when it says a match is possible are the rules consulted one by one, to say *which* rule matched (for `-debug`).

The cost is about 9 μ s per entry, and a property test checks that the quick path never changes an answer.

4.5 Loading a rule file

A rule file is written by a person in an editor, and a deny rule that loads but never matches is the worst outcome. The parser therefore either normalizes or refuses. It strips a leading byte order mark (which otherwise broke the first rule) and accepts any line ending (a bare carriage return joined lines). It refuses, naming the line: an inline comment, `~name`, a quoted pattern, an invisible character, a lookalike of the slash, negation in the deny file, and the malformed patterns of the functional specification (RUL-12). Whatever loads, works as written.

4.6 The built-in rules

The core deny rules are compiled into the binary, so they apply even without a deny file. The credential rules are not anchored to the current home folder, because the same folders exist in backups, clones, other accounts' homes and under a wrong `HOME`. If a deny file exists it is authoritative, and `CoreMissing` compares the core rules with the file's rules *as names* to decide which are not in effect. The list of credential locations for the hard-link index (Section 3.9) mirrors the credential entries of the core rules.

5 The HTTP Layer

5.1 Middleware

The middleware is a single function that either refuses a request or passes it on with the prefix removed. Its checks, and the order of them, are in the life of a request (Section 3); this section records the reasons.

Host. A page on another name that resolves to the loopback address still sends its own name in the `Host` header, so an exact allowlist of the two loopback names with the port defeats DNS rebinding. The port is fixed at start.

Origin, Referer, Fetch metadata. A browser serializes `Origin` as exactly scheme, host and port; anything else (a path, credentials, an opaque `null`) is refused rather than interpreted. `Sec-Fetch-Site` must be `same-origin` or `none` (or absent, for clients that do not send it). The value `same-site` is refused because a page on another port of the same host is same-site to the browser, so `SameSite=Strict` does not stop it.

Read-only. Only `GET` and `HEAD` reach a handler; the endpoint table contains no other method.

5.2 The session and the prefix

The launch token and the session value are 256-bit random values, compared in constant time. The token is exchanged once, atomically, for the session cookie; the response redirects to the prefixed root so the token leaves the address bar and the history.

A cookie is scoped by host and by path, never by port. A plain `Path=` cookie would be sent to every other web server on 127.0.0.1 that the same browser visits, and a server that received it could replay it. The design therefore serves everything under a random 128-bit path prefix chosen at start, sets the cookie's path to it, and refuses (403) any URL not under it. The browser then sends the cookie only for URLs under the prefix, which no other server has. The prefix is a scope, not a secret: the session cookie is still required. The front end forms every URL relative to its own location so that this needs no configuration.

5.3 Headers and framing

Every response carries the headers of SEC-9. Two endpoints relax framing, deliberately and narrowly: the fixed Markdown page and the PDF may be framed by the interface's own pages (`X-Frame-Options: SAMEORIGIN, frame-ancestors 'self'`), and the interface itself may not.

5.4 Streaming

Listings and searches are newline-delimited JSON, flushed as batches are ready, because a folder can hold hundreds of thousands of entries and the first screen must not wait for the last. A listing that breaks after it has begun ends with an error line rather than a status code, since the status was already sent. Pagination was rejected: the client sorts, and a sort needs all entries, so streaming keeps the whole listing available and still paints early.

5.5 Errors

One function maps every guard error to a status. Denied, outside-the-roots, nonexistent, malformed and not-regular all map to the same 404 with the same body; only `-debug` adds the rule name. Permission errors on paths that passed every rule map to 403; cloud-only files to 409; wrong kinds to 415; anything unexpected to 500 with the detail in the log only.

5.6 Streams end explicitly

A listing and a search are streamed as newline-delimited JSON, and a response that simply stops (a dropped connection, a server error after the header) would otherwise look like a folder that was just short. Both therefore end with a line that says how they ended: a listing with `{"done":true}`, a search with `{"done":true,...}` carrying how many entries were visible, whether a limit stopped it, and whether part of it could not be searched (`incomplete`); an error mid-stream is a line of its own instead. The client requires the done line and treats a stream that ends without one as broken, not as complete.

5.7 Timeouts and shutdown

The server sets a header-read timeout against slow clients and has no write timeout, because streams may legitimately last. On interrupt or termination it stops accepting requests and gives requests in progress three seconds.

6 Content-Typed Endpoints

Three endpoints look inside a file: inline images, inline PDFs, and archive listings. Each is defined by a test on the file's *leading bytes*, never its name, and each opens the file through the guard, so every property of Section 3 applies to it unchanged. A fourth, Quick Look pictures, is different in kind: macOS reads the document, not **fsb** (Section 6.3).

6.1 Images

An image is served inline only if its first bytes are a PNG, JPEG, GIF or WebP signature. The response is sent with a policy that sandboxes it and allows no resources, so even a crafted file has nothing to reach. SVG is never accepted, because it can carry script, and neither is HTML. A file named `.png` that is not one is refused (415) and the interface falls back to the ordinary file preview.

6.2 PDFs

A PDF is served inline only if it begins with `%PDF-`. It cannot be sent with the **sandbox** policy that images get, because browsers then refuse to display it. The protection is instead the byte check, framing headers that allow only the interface's own pages to embed it, a policy that forbids script in the response, and the browser's own PDF viewer, which runs in an isolated component of its own. The interface asks first, with a one-byte range request, whether the file really is a PDF, and only then points a frame at it.

A viewer may take the keyboard focus by itself (Chrome's does, for an encrypted PDF's password field), and once focus is inside the frame no key event reaches the interface. The preview pane records whether the pointer is over the frame and whether the last key was Tab; when the window loses focus to the frame with neither, focus goes back to the list (PRV-8).

6.3 Quick Look pictures

For iWork and Office documents the preview is the PNG that Apple's `qlmanage -t` draws, which is what Finder shows. This is the one place **fsb** hands a file to another program, and that program opens what it is given by itself, resolving more than the guard can see: a Finder alias is a small regular file, marked by its `FinderInfo` attribute, that Quick Look follows to its target. An independent review showed that an alias named `report.docx` made Quick Look draw a file in a denied folder, and one outside the roots. Quick Look is therefore never given the user's file.

1. **A private copy.** `guard.CopyForQuickLook` opens the document exactly as every endpoint does (roots, deny rules before and after, hard links to denied files), requires an iWork or Office extension on the real path, refuses a cloud-only document, and copies the bytes, from the descriptor it verified, into a fresh temporary folder that only the user can read. A copy has no extended attributes and no resource fork, so an alias's copy is just its bytes, which Quick Look draws, at most, as a blank page; and nothing can be swapped between the check and the drawing.
2. **Packages.** A package document is walked without following links. Every file in it is opened through the guard by its full path, and must still lie inside the package (a folder swapped for a link after the walk listed it leads elsewhere, and is caught here). A part that is denied, a link, a hard link to a denied file or not a regular file is left out of the copy, as a listing leaves it

out, so the answer does not reveal it (the same review found that refusing the whole package did); a cloud-only part refuses the package.

3. **Bounds.** A copy is limited to 256 MB and 5,000 entries, and to 20 seconds, and it stops when the browser abandons the request. The time limit is *cooperative*: the reader checks the request's context before each read and again when a read returns (so a copy that ran out of time while a read was blocked fails rather than reporting success when that read reports the end of the file), but it cannot interrupt a read the file system is itself holding up, such as a stalled network volume. The slot is held until that read returns. Enforcing a hard limit would need the copy to run somewhere it could be abandoned and cleaned up, which this version does not do; abandoning the work in the background while releasing the slot was rejected, because the work would still be running. Copying and drawing share two slots. `qlmanage` never finishes on a document it cannot draw, so it runs under a 5-second deadline and is killed when the browser abandons the request. Exactly one PNG under 20 MB must come out, or there is no picture. The picture is served like an image (sandbox policy, `nosniff`) and never cached, and the temporary folder is removed.

Choosing by extension rather than by bytes (PRV-2) is deliberate: Quick Look decides what the document is, and a package is a folder with no bytes of its own. The extension only decides whether to ask. Keynote's `.key` stays covered by the core rule `*.key`.

6.4 Archives

An archive listing returns names and sizes only, never member data, and nothing is extracted. The format is decided from bytes: the zip signature, the `ustar` marker at offset 257 of a tar, or a gzip stream that contains a tar. The limits of LIM-1 bound a hostile file:

- A zip's directory is read by the guard itself, record by record, rather than by the standard library, which would build the whole directory in memory before any limit could count it. Reading stops at 200,000 entries or once 64 MiB of directory have been read, and the listing says so; the byte total is checked between records, so the last record may cross it by at most its own length (a bounded overrun, not an exact limit); a directory whose own count disagrees with what it holds is refused. A zip64 end-of-directory locator is honored whenever it is present (a first version compared a 16-bit count with 200,000, a test that could never be true). A fuzz test compares this reader with the standard library's wherever both accept an archive; it does not require that every archive the standard library accepts is accepted here, which is listed among the gaps in Section 13.
- A plain tar is read through a wrapper that checks the request's context but keeps the file's `Seek`, so that the tar reader skips a member's body instead of reading it (a first wrapper exposed only `Read`, and listing an 8 MiB member read all 8 MiB). A gzip-compressed tar is streamed through a reader capped at a number of decompressed bytes, so a bomb costs a bounded amount of time and memory; a listing that ends because the budget was spent exactly at a member boundary says it was cut short. All archive listing stops when the request is abandoned.
- Member names are untrusted: control characters, Unicode formatting characters (such as bidirectional overrides) and invalid UTF-8 are replaced, a name is cut after 4,096 bytes and an ellipsis added (so at most 4,099 bytes are shown), and names are only ever displayed, never

used as paths.

6.5 Rendered Markdown

Rendering a Markdown file must not run code, load anything remote, or reach the interface. It is done in three layers, each assuming the one before failed.

1. **Escaping.** The Markdown is converted by a library configured so that raw HTML in the source becomes text; links carry no address at all (an attribute says what they mean: a file on the disk, or a web address); images carry no source (a marker names a local file); a remote or unsupported image becomes a text placeholder. A link is resolved against the Markdown file's folder to a normalized absolute path, or dropped to plain text if it is not a relative path or an `http` or `https` address.
2. **Sanitizing.** The result goes through DOMPurify with a fixed list of allowed tags and attributes.
3. **A sandboxed frame.** The sanitized markup is sent by `postMessage` to a fixed page, framed with `sandbox="allow-scripts"` (so it has an opaque origin and no access to the session or the interface). The page's own policy is `default-src 'none'` with images limited to `data:`, and admits its one script and one style *by hash*, computed from the page at start so that a change to the page cannot silently invalidate the policy. The page accepts messages only from its parent.

Links are not followed inside the frame. A click sends a message to the interface, which validates it and either changes the location to the target file's folder with the file selected, or opens a web address in a new tab without a referrer. Images inside the roots are fetched by the interface, through the guarded image endpoint, and passed in as `data:` URLs, because the cookie's scope and `SameSite` would otherwise stop the opaque-origin frame from fetching them. Opening a Markdown file therefore makes no request to anything outside the machine. The interface fetches at most four images at a time, refuses one whose declared length is over 5 MB before reading it, and otherwise counts the bytes as they arrive and cancels the request the moment they pass 5 MB, because a declared length is not trusted (`fetchcap`).

7 The Front End

7.1 Structure

The interface is a single page built with Svelte 5 (runes), TypeScript and Vite. Its logic is kept in small *pure* modules, which import nothing from the browser, so that they run under Node's own test runner: ordering (`sort`), filtering (`filter`), column layout (`columns`), preferences (`prefs`), preview classification and text helpers (`preview`), the path and location functions (`format`, `goto`), the key mapping (`keys`), the Markdown conversion (`markdown`), and where the page was loaded from (`base`). The components hold only what needs the DOM.

7.2 Decisions

Virtualized list. Only the rows on screen are drawn, from a scroll position and a fixed row height; a folder of 100,000 entries draws about 30 rows.

Location as state. The current folder, and the entry to select, are in the URL fragment (`#/path?select=name`), which is never sent to the server, so folders are bookmarkable and Back works without any routing library. Parsing is strict: a bad escape is not silently replaced.

Requests relative to the page. Every request is formed from the page's own directory, so the random prefix (which scopes the cookie) needs no configuration.

Sorting in the browser. The server streams entries in the order the file system gave them; the browser sorts, with a comparison that is a total order and does not depend on the input order (a test). The collator used for names compares digits by value, so `file1` and `file01` compare equal; the final tiebreak is the raw characters, so they still have a fixed order. A time that cannot be parsed sorts as the oldest, so the order stays total.

Names are displayed, not trusted. Characters that would disguise a name are replaced by visible markers for display only. Names are always inserted as text, never as markup.

Preferences are optional. They are kept in local storage with every read guarded, and every value falls back on its own. Local storage is per origin, which includes the port; that is why the server remembers its port (Section 2.5).

The pane grows with the window. `clampPaneWidth` bounds the preview pane between 240 pixels and the window's width less `RESERVED_FOR_LIST` (320 pixels, matched by a CSS `max-width`), and the page tracks the window's width live, so the pane can widen with the window and is narrowed if the window shrinks. An earlier fixed ceiling of 900 pixels was reported by the user as a usability limit, not found by a test.

Every letter types ahead. Search, Go to path and Copy are on Alt+S, Alt+G and Alt+C, read by physical key (`KeyboardEvent.code`) because macOS changes the character Option produces; so no plain letter is reserved and type-ahead (`KEY-3`) can reach every name.

A click selects. A plain left click on a file name calls `preventDefault`, so it selects the row instead of following the link's `download` attribute; modified clicks and the context menu still reach the browser's own download (`MOUSE-1`).

A search belongs to one run. Starting, replacing or clearing a search aborts the old one and clears its timers, and everything the old run still delivers (batched results, its end, its error) is checked against the current run before it touches the page, so a slow old search can never overwrite a new one's results. The logic is a small module (`searchrun`) tested without a browser. A stream that ends without its done line is an error, and the done line's `incomplete` produces a note.

A resize handle keeps its keys. When a column or the preview pane's edge has the focus, its arrow keys resize it and the global key handler ignores the event, so resizing does not also move the selection or leave the folder.

No dynamic code. There is no use of `eval`, of `innerHTML`, or of any other markup sink except one: the output of the syntax highlighter, which escapes its input.

7.3 The preview pane as a state machine

For a selected entry the pane is in one of a few states (idle, loading, folder, image, Quick Look picture, PDF, archive, text, binary, empty, cloud-only, error). A file name gives a *hint*, the server decides from the bytes, and a hint the server rejects falls back to the ordinary head request, which never falls back again, so every run ends in a terminal state after at most two requests. (A Quick Look picture that fails falls back to that head request, or, for a package document, which lists as a folder, straight to the folder state.) Every request started for a selection, including a fall back, shares one cancellation handle and is cancelled when the selection changes, so a late answer can never be shown for the wrong file. The complete transition table (states, events, next states and the request each transition starts) and the laws are in the algebraic specification, in its section on the preview state machine. The code keeps the state in one variable set where each answer arrives, rather than in a table of its own.

The Details panel (metadata and extended attributes) is a second, independent request against the same selection, and keeps its own error state rather than sharing the main preview's. An earlier version used one shared variable for both: whichever request settled last silently overwrote the other's message, and the panel's own error was suppressed whenever the main preview had already failed, which could leave it reading "Loading..." forever even though its own request had already come back (Finding 18).

The folder listing has its own, separate error mapping (`describe` in `App.svelte`), independent of the preview pane's, and it had never been given a case for status 409: a cloud-only folder (Finding 19) fell through to the generic "Server error (*n*).", instead of the same cloud message the preview pane already shows for a cloud-only *file* (Finding 20). Two error mappings for the same status code, one of them incomplete, is the same shape of gap as Findings 4 through 6: a case a status code can take that nothing had exercised. Two independent outcomes need two independent variables.

7.4 Hover peek

A short delay before the request, a cache, a limit on simultaneous requests, and cancellation on pointer movement, scroll or key press. It requests the same head endpoint as the preview, so it is subject to the same rules, and is off if the user turns it off.

8 Verification

The strategy follows the principles: the parts that decide what is served get the heaviest testing, and the tests that matter most let the operating system decide the facts.

8.1 Layers

Layer	What it establishes
Unit and integration (Go)	Behavior of the matcher, the guard, the middleware and every endpoint, including deliberately hostile paths. A test fails the build if an HTTP-facing package imports <code>os</code> , <code>io/fs</code> , <code>syscall</code> or <code>path/filepath</code> .
Property tests	The laws of the algebraic specification, with fixed seeds: monotonicity and order independence of deny, ancestor closure, fold invariance, listed implies openable, found implies listed, and the equivalence of the quick matching path with the plain one.
Fuzzing	The guard (with an oracle that does not trust <code>fsb</code> 's rules: whatever it returns must not be the same inode as a known secret or contain a secret marker), the matcher, the middleware, and the whole server stack.
Differential test against the OS	For each secret, thousands of generated spellings (case, folding aliases, decomposed accents, separators, dot segments, firmlink and symbolic-link aliases, named forks) are filtered by the operating system to those that really reach the secret, and every one must be refused. The reverse direction requires every such spelling of an ordinary file to be served. Reverting the folding fix makes it fail about 1,500 times.
Real-volume tests	Root containment on a real case-sensitive volume, run when a location for one is given.
Front-end tests (Node)	The pure modules, including the laws for ordering, filtering, location round trips, link resolution and name display.
Browser tests	A dependency-free suite written once against a driver interface, run with real keyboard and mouse events in Chrome (DevTools protocol) and Safari (WebDriver). It includes attacks from a second origin and from a page on another port, a DNS-rebinding hostname, the cookie's scope, and a 100,000-file folder.
Mutation checks	Removing a rule or reverting a fix must make a test fail; this was done for the folding fix, the cookie path, and others.
Algebra	The laws, and the defects found by stating them (Section 10).

8.2 Continuous integration

On every change: formatting, vetting, the Go tests under the race detector, the front-end tests, type checking, a build whose output must equal the committed one, an audit of the packages that ship, the Chrome browser tests, and timed fuzzing of the four fuzz targets. A scan for known vulnerabilities in the Go modules runs as well. The Go tests run on macOS and Linux; the rest on Linux. The workflow runs on GitHub's hosted runners for every push to a private repository; its first run (2026-10-02) found finding 28 of Section 10, and every run since has passed.

8.3 What testing does not establish

None of this is a guarantee. It is a record of what was checked. A fuzzer reaches only what its fixture contains, and an early version missed a bug for that reason; the differential test exists because reasoning about names had been wrong. Findings 17 through 20 (Section 10) were caught by none of the automated layers: no fixture anywhere contained a UNIX domain socket or a real File Provider placeholder folder, so nothing had ever exercised the `errno` either produces, and the Details-panel race needed a preview that fails, which no test happened to select. All four were found by using the running application, not by any automated layer, which is itself a reason to keep doing that rather than treating the test suite as the last word; Finding 21 was found while fixing

them, by reading the surrounding code rather than by any test. Independent reviews were done by an assistant (Section 13) in two sequences: in September, of Quick Look, background mode and process control; and in October, of the whole code, as an external review, then of its fixes (which found two further hard-link bypasses), then of those fixes (which found seven defects, several in the fixes themselves), and finally of that round, which found nothing new (Section 10). A professional penetration test was not done.

9 Decisions and Alternatives

Each row is a decision that shaped the design, with what else was possible and why it was not chosen.

Decision	Rejected	Why
Write from scratch in Go with an embedded Svelte front end	Fork an existing multi-user file manager	The existing one is built around a jailed root, configuration in its own UI, and create, rename and delete. The security model here (whole home, read only, two-tier rules) is different in kind, and a fork would carry code paths that must not exist.
Two tiers: hide and deny	One list	Hiding clutter is cosmetic and must not be a security control; blocking must be absolute, not overridable, and load only from the global file so that a folder cannot relax it. Deny wins over ignore.
Deny loads only from the global file	Per-folder deny files	A per-folder file could be planted in the tree being browsed to weaken protection.
Core rules built in and active by default	Ship them commented out, with a warning	A tool that exposes <code>~/ssh</code> until configured is unsafe by default. The warning now covers only the case where the user weakened them.
Bind loopback only, not configurable	A flag for other interfaces	There is no safe way to serve the whole home folder to a network, so the option is not offered.
Single-use launch token, then a cookie	A permanent token in the URL	A token left in history or a shell must be useless.
Path prefix scoping the cookie	A plain cookie; a custom header on every request	Browsers cannot scope cookies by port. A header cannot accompany images, frames, PDFs and downloads. A random path scopes the cookie exactly.
Identity for hard links, from a background index	A hash of contents; a synchronous walk	A hash flags harmless copies, reads every candidate, and answers a different question. A synchronous walk did not finish in four seconds on a real home folder and would stall a request. Refusing until known keeps it safe meanwhile.
Roots decided by identity	Comparing folded text	Right on a case-insensitive volume, wrong on a case-sensitive one.
Full case folding, from <code>x/text</code> , applied to rules	<code>ToLower</code>	The volume uses full folding; simple folding made some spellings of a denied name invisible to the rules.
Refuse a file whose kind the bytes do not confirm	Trust extensions	An extension is chosen by whoever named the file.
Markdown in a sandboxed frame, sanitized, with a hash-pinned policy	Render in the page with a sanitizer only	One layer failing must not compromise the session; the frame has no origin, no network and one fixed script.

Decision	Rejected	Why
PDF in the browser's viewer in a frame with narrow framing headers	Bundle a PDF library; sandbox the frame	A library is large and a second parser to defend; a sandboxed frame is refused by browsers.
Stream listings as newline-delimited JSON	Pagination	The client sorts, which needs all entries; streaming paints early and keeps them all.
Sort in the browser	Sort on the server	The sort key and options change often; the total-order comparison is a pure, testable function.
Pure modules tested with Node's runner	A browser test for every function	Fast, dependency-free, and the laws are stated on them.
Commit the compiled front end	Build it in the Go build	Building the binary must need no Node. CI checks that the committed output is current.
Deny checks over-match; allow checks decide by identity	One equality for both	The two directions fail differently: over-matching a deny only blocks more, over-matching an allow serves the wrong folder.
Serve a multi-name file only when every name is accounted for, now	Allow until known; allow a file the walk cannot see after a second walk; trust a file whose change time is old	The window in which a hard link to a secret is served must be zero, at the cost of some unavailability. A file's own change time does not move when a folder holding one of its names is renamed, and a fail-open rule for a file the walk could not see let a link through; each was tried and broken by a review.
A listing or search reuses a hard-link verdict for one second	Re-verify on every entry; remember for the whole response	Re-verifying costs a look at every recorded name per entry (the square of the link count for a store with thousands of names). Remembering for the whole response lets a rename that leaves the count unchanged stay invisible for as long as the response runs. One second bounds the staleness; opening a file never uses the memo.
Cancel copying and listing cooperatively	Abandon the work and release the slot; promise a hard deadline	A context check cannot interrupt a read the file system is holding up. Abandoning the work would release the slot while it still ran, so the limit would be a fiction. A hard limit needs work that can be killed and cleaned up, which this version does not have; the limits are described as cooperative.
Read a zip's directory ourselves, record by record	The standard library's reader behind a count check	The library builds the whole directory in memory before any count of ours applies, and a check on a 16-bit field cannot see a zip64 count. A limit has to be applied while reading.
End every stream with a done line	Treat the end of the connection as the end	A dropped connection is indistinguishable from a short folder; the line says how it ended, and the client treats its absence as an error.
Hand-written manual page in roff	Generate it from Markdown	No build dependency, and the format users expect.
Remember the last port; <code>-port</code> for one run only	A preferences file or endpoint on the server; a cookie; <code>-port</code> that is remembered	A server-side store needs a write path, breaking read-only by construction, and would be one file shared by concurrent instances. Cookies are not scoped by port, so one would either reset like local storage or reopen the cross-server leak the path prefix closes. A remembered <code>-port</code> would turn a one-off workaround into a permanent move to an empty set of preferences.

Decision	Rejected	Why
Fail, naming the holder, when the remembered port is taken	Silently pick another port	A silent move looks exactly like losing every preference, with nothing to say why.
Background mode by re-executing in a new session, with a hand-off pipe	Tell users to run fsb & ; daemonize with a double fork	Job control is unfamiliar to many users and ties fsb to the terminal. Go cannot fork safely; a re-executed child that reports its own startup over a pipe lets the parent show the real error and leave nothing running.
Identify a running fsb by owner, executable file and start time	By the command name ps shows	On macOS that name is the process's argv[0] , which any program can set (found by the independent review).
Quick Look draws a private copy	Hand Quick Look the user's path, re-checking afterwards	Quick Look resolves Finder aliases that the guard cannot see, so a path that passed every check could still lead it to a denied file (found by the independent review). A copy made from a verified descriptor has nothing to follow and cannot be swapped.
Quick Look chosen by extension	Sniff the bytes, as other previews do	Quick Look decides what a document is, and a package has no bytes of its own; the copy makes the name harmless.
Leave denied parts out of a package's copy	Refuse the whole package	Refusing answered differently for a package with a hidden part than for a clean one, revealing what a listing hides.

10 Limitations, Risks and History

10.1 Risks

R1: sensitive paths are exposed by default. The root is the home folder, so without protection `~/ssh` and similar would be reachable. Mitigated by the core deny rules, active by default and built in, and by a startup notice and a banner if the user weakens them. Residual: a secret with an unremarkable name outside the core list, the optional rules left off, and a user who disables the core rules and ignores the warning.

R2: files with several names. Path rules cannot see a second name for a file. Covered for denied names under the roots and for the credential locations, by refusing any file whose names are not all accounted for; see the limits below.

R3: untrusted content in a page that holds a session. Markdown, PDF and archive names are rendered or shown; mitigated by the layers of Section 6.

R5: another program reads the user's documents. Quick Look draws iWork and Office documents with Apple's generators. Mitigated by giving it only a private copy of what the guard would serve, with bounds on size and time; residual: a flaw in a generator, as when Finder shows the same picture.

R4: permission prompts. macOS asks permission the first time for Documents, Desktop, Downloads and external volumes; the documentation says so.

10.2 Known limitations

- A denied name outside every root and every protected credential location is not found as another name of a file, so a file with such a name is refused (a pnpm store outside the root, or a clone made with `git clone -local`, can make ordinary files unavailable). A listing or a search may show a name or a size up to one second after the file was renamed onto a denied name; opening the file never does. Cancellation of copying and listing is cooperative: a read that the file system is itself holding up cannot be interrupted, and keeps a Quick Look slot until it returns. Search completeness does not include cloud-only folders, which are skipped on purpose.
- Response timing is not equalized: a denied path can be answered slightly faster than a missing one. The status and body are identical.
- Anything that runs as the user can read the same files. `fsb` is not a sandbox against local malware.
- The metadata endpoint reads a symbolic link's text from the requested path after the file was opened; an attacker who could rewrite that link during the call could learn the text of a denied path (not its contents). Requires a same-user attacker.
- Rule files are read at startup only.
- `-stop` would take another program of the same user for `fsb` if that program's executable file were itself named `fsb`.
- Not yet tried with real cloud-placeholder files or in Firefox; a real mouse drag of a column header in Safari is unverified.
- The browser test for Safari occasionally fails in a run of tests that need a focused window, and passes on rerun.

10.3 History of defects found by the design's own checks

Stating the laws of the algebraic specification, auditing every comparison of names and paths, and using the running application found thirty-one defects, all fixed and each covered by a test where the underlying condition could be reproduced at all (Section 13 says where it could not). In brief:

1. Simple case folding let `.fbh` open `.ssh` (a bypass of a deny rule).
2. A plain file named like a folder-only rule was listed but could not be opened.
3. A negated glob class could match across a path separator.
4. A row order that was not transitive when a time could not be parsed.
5. The filter and the search disagreed about the sharp `s`.
6. A stale fallback request could show the wrong file's preview.
7. Firmlink prefixes were matched by byte length (a bypass under a root of `/`).
8. Root containment compared spellings (wrong on a case-sensitive volume).

9. Breadcrumbs, selection and the core-rule warning compared names by spelling.
10. The credential rules were anchored to one home folder.
11. A hard link to a credential file was served.
12. The session cookie followed the host, not the port.
13. A hard link to any other denied file was served.
14. Rule files that loaded without error and did nothing.
15. Secrets under other names (`id_rsa`, certificate stores, password databases).
16. File names that disguised themselves with bidirectional characters.
17. Opening a UNIX domain socket used an `errno` (`EOPNOTSUPP`, or `ENXIO` on Linux) that the guard's error map did not recognize, so it surfaced as an opaque 500 instead of the same "not found" every other special file gets. Found from a screenshot of the running application, not from any test.
18. The preview pane's Details panel shared one error variable with the main preview and hid its own message whenever the main preview had already failed, so it could read "Loading..." forever even though its own request had already finished. Found alongside the previous defect, in the same screenshot.
19. Listing or searching a folder that is itself a cloud-only placeholder (a File Provider domain such as OneDrive or iCloud Drive can make a whole folder dataless, not only the files in it) went straight into reading its directory entries, which the provider's virtual file system answered with something the guard's error map did not recognize, surfacing as an internal error instead of the same cloud message a dataless *file* already gets. Found from a screenshot of a real OneDrive folder, not from any test: no fixture anywhere had a real cloud-only placeholder, only a synthetic flag on individual files.
20. The folder listing's own error mapping, separate from the preview pane's, had no case for status 409, so once the previous defect was fixed at the guard level the interface still showed "Server error (409)." for a cloud-only folder instead of explaining it. Found alongside the previous defect.
21. While fixing the above, a latent defect in **Search**: a directory reopened from its queue that turned out no longer to be a directory (a rare race) left its descriptor open, because the code path that skipped it predates the fix and never closed what **Open** had just handed it. The new dataless check would have turned this from a rare race into a routine leak, since a cloud-only subfolder hits the same code path on every walk that reaches one.
22. Found by the user: a root given twice (`-root ~` when the home folder is already the default root) was served twice, and the interface, which keys its list of roots by path, failed so that every folder looked empty. Roots are now served once, compared by identity.
23. Found by the user: Chrome's PDF viewer moves the focus to the password field of an encrypted PDF, after which no key reached the interface, so arrowing through a folder stopped dead on that file. Focus now returns to the list unless the user clicked or tabbed into the viewer.

24. Found by the second independent review (and confirmed): a Finder alias named like an Office document made Quick Look draw the alias's target, which could be denied or outside the roots. Quick Look now draws a private copy (Section 6.3).
25. Found by the same review: refusing a package document that held a denied part answered differently from a clean package, revealing what a listing hides. Such parts are now left out of the copy.
26. Found by the same review: `-stop` took any process whose `argv[0]` ended in `fsb` for `fsb`. It now checks the owner, the executable file and the recorded start time (Section 2.5).
27. Found by the same review: `fsb`'s own files in `~/config/fsb` were written through symbolic links. They are now opened without following links.
28. Found by the first run of continuous integration: the module named Go 1.26.0, whose standard library has since had security fixes (reported by `govulncheck`); the developer's machine happened to use a newer release. The module now names the toolchain (Go 1.27.1). The same run found two tests that assumed macOS.
29. Found by an external review (October 2026, eleven findings, all confirmed and fixed; the commits cite them by number). The ones that mattered most: a file with several names that the index had not seen was *allowed* during the rebuild cooldown (the code returned the wrong flag, and the tests, which set the cooldown to zero, hid it); the index trusted names recorded at the last walk; a zip's entry limit compared a 16-bit count with 200,000, so it could never trip, and the standard library built the whole directory first; the index walk and the Quick Look package copy entered cloud-only folders; a symbolic link to `/` passed a check that only looked at spellings; an image fetch read its whole body before checking its size; search's work limit ignored entries it excluded; a tar.gz stopped exactly at a member boundary looked complete; an old search's timer overwrote a new search's results; `resize-handle` keys also moved the selection; and a collator that compares digits by value left `file1` and `file01` in an order that depended on the input.
30. Found by an independent review of those fixes: two more hard-link bypasses and a quadratic cost in the index, and zips written to a pipe were refused. Fixing them with a change-time test was itself broken by the next review (renaming a folder renames its files without touching them), which led to the policy of Section 3.9: serve a file only when every name is accounted for, now. A third round refined it (names folded by parent identity; one retry walk).
31. Found by a second review of the whole (seven findings, all medium): a hard-link verdict remembered for the length of a listing or search hid a rename that kept the link count; a folder queued for search could be swapped for a symbolic link before it was entered; the one retry walk a short-counted file is owed was spent by a lookup during the cooldown that could not start it; the wrapper that added cancellation hid `Seek`, so listing a plain tar read every member's body; a copy that ran out of time while a read was blocked reported success when the read returned end of file; a folder that could not be opened left a search looking complete; and an empty extended attribute that grew between two system calls was sliced out of an empty buffer and panicked. A third review confirmed each fix and found nothing new. Two choices came out of it and are stated as limits rather than hidden: the one-second reuse of a verdict, and cooperative (not hard) cancellation.

The lesson common to several: name equality is a property of the volume, so deny checks may over-match but allow checks must decide by identity. Findings 17, 18, 19 and 20 are a different lesson: an error path that no test happened to reach (no fixture had ever contained a real socket, a real cloud-only folder, or a selected file whose main preview fails) is still worth trying by hand. Findings 29 to 31 are a fourth: a fix can introduce the next defect (a wrapper that hid **Seek**, a retry spent before it ran, a memo that outlived the facts it summarized), so each fix got a test that fails without it, and the reviews were repeated until one found nothing. Finding 24 is a third: handing a path to another program hands over that program's idea of what the path means, which the guard cannot check; the safe input is bytes the guard has already read.

10.4 Milestones

Milestone	Scope	Status
M0 Skeleton	Repository, CI, guard, matcher, middle-ware	Done.
M1 Crawl	Browsing, listing, filtering, sorting, rules	Done; 100,000-file folder measured. Daily use by the author still to confirm.
M2 Walk	Preview, hover peek, search, attributes, columns, rendered Markdown, PDF, archives	Built and tested in Chrome and Safari; not yet on real cloud placeholders or in Firefox.
Hardening	The audits and defects of the preceding subsection	Done, including three independent reviews in October 2026 (the last found nothing new) and continuous integration on macOS and Linux. A professional penetration test was planned but, by the owner's decision, not done.
M3 Public release	Public repository, documentation, a source release	Done: version 1.0 (October 2026), source only; no binaries and no Homebrew formula, by decision.
M4 Run	The stage-three requirements of the functional specification	Not planned: the project is finished and provided as-is.

11 Tested Environment

Every result in this document, and every test referenced by it, was produced on the single machine described below, except that continuous integration also runs the Go tests on macOS and Linux and the Chrome end-to-end suite on Linux, on GitHub's hosted runners. Nothing else has been run on any other machine, and nothing at all on the browsers this lists as not installed. Section 13 states plainly what that leaves unchecked.

11.1 Hardware

Model	MacBook Pro
Chip	Apple M2 Max
Cores	12
Memory	64 GB
Architecture	arm64

11.2 Operating system and browsers

macOS	27.0.1 (build 26A434)
Safari	27.0.1
safaridriver	Included with Safari 27.0.1 (22625.1.29.11.28)
Google Chrome	Google Chrome 154.0.8037.93
Firefox	not installed; not tested (DST-4 note; Section 13)

11.3 Go toolchain and modules

Toolchain	go version go1.27.1 darwin/arm64
GOOS/GOARCH	darwin/arm64
CGO_ENABLED	1 (the program uses no cgo; clang is listed below only because the toolchain can call it)

Resolved module versions (`go list -m all`), exactly as recorded in `go.sum`:

Module	Version
golang.org/x/mod	v0.41.0
golang.org/x/sync	v0.23.0
golang.org/x/sys	v0.48.0
golang.org/x/text	v0.42.0
golang.org/x/tools	v0.49.0

11.4 Front end: Node and npm packages

Node.js	v26.10.0
npm	11.19.1

Installed package versions in `web/` (`npm ls -depth=0`), which match the versions pinned in `web/package-lock.json`:

Package	Version
@sveltejs/vite-plugin-svelte	7.3.1
dompurify	3.4.16
highlight.js	11.12.0
marked	18.0.14
svelte	5.57.1
svelte-check	4.7.6
typescript	6.0.3
vite	8.3.1

11.5 Build and documentation tools

<code>make</code>	GNU Make 3.81
<code>git</code>	git version 2.54.0 (Apple Git-157)
<code>clang</code> (Xcode Command Line Tools)	Apple clang version 21.0.0 (clang-2100.3.34.2)
<code>mandoc</code>	bundled with macOS 27.0.1 as <code>/usr/bin/mandoc</code> (BSD mandoc; the tool exposes no version flag)
<code>L^AT_EX</code>	pdfTeX 3.141592653-2.6-1.40.29 (TeX Live 2026)
<code>latexmk</code>	Latexmk, John Collins, 9 March 2026. Version 4.88

The L^AT_EX packages loaded by the shared preamble (`specs/common/preamble.tex`) are: `fontenc`, `inputenc`, `lmodern`, `geometry`, `microtype`, `parskip`, `amsmath`, `amssymb`, `amsthm`, `mathtools`, `booktabs`, `longtable`, `array`, `enumitem`, `needspace`, `xcolor`, `listings`, `fancyhdr`, `hyperref`, `bookmark`, `xurl` and `tikz`, all from the TeX Live distribution named above.

11.6 Regenerating this section

This section is generated, not written by hand. On the machine that ran the tests:

```
specs/tools/tested-environment.sh > specs/design/sections/appendix-environment.tex
make -C specs design
```

12 Requirement Traceability

Every requirement of the functional specification is listed below with the tests that check it, cited by name so they can be run and read. A citation without a file extension names a Go test function (`internal/..._test.go` or `cmd/fsb/..._test.go`); `e2e`: names an end-to-end browser test by its title in `e2e/suite.mjs`; `web`: names a Node test file under `web/src/lib/`. Where a requirement is not checked by an automated test, that is stated plainly rather than left to be inferred from an empty cell; Section 13 collects the reasons. This table was produced by reading the requirements against the test suite as it stands on the commit this document was built from; it is not generated, and it should be re-checked whenever a requirement or a test changes.

Req.	Coverage
CLI-1	The end-to-end harness starts the real binary with no path argument for every browser test (<code>e2e/lib/server.mjs</code>); <code>TestResolveRoots</code> .
CLI-2	<code>TestResolveRoots</code> , <code>TestNewValidatesRoots</code> . Symbolic-link resolution of a root is covered generally by <code>guard.New</code> 's use of <code>EvalSymlinks</code> , not by a root-specific test.
CLI-3	<code>TestResolveRoots</code> (repeated <code>-root</code>); <code>TestNewDropsDuplicateRoots</code> (the same folder by several spellings, served once).
CLI-4	<code>TestResolveRoots</code> (<code>/</code> , <code>-root /</code> , and with the flag); <code>TestSystemRootOptInAppliesToResolvedRoots</code> (a symbolic link to <code>/</code> is refused too, by identity). The exact wording of the refusal is not asserted by a test; verified by hand (2026-09-21).
CLI-5	<code>TestOpenCommand</code> , <code>TestCheckBrowserName</code> , <code>TestRunRejectsABadBrowserNameEarly</code> . <i>Not automated</i> : that a browser which fails to open leaves the server running (verified by hand once, 2026-09-21).

Req.	Coverage
CLI-6	<code>TestDebugRevealsRuleOnlyForDeniedPaths</code> , <code>TestDebugNamesTheRuleOnNewEndpointsToo</code> (the underlying behavior, via <code>server.Config.Debug</code>). The <code>-debug</code> flag's wiring in <code>cmd/fsb</code> is not itself exercised by a test.
CLI-7	<code>TestInitWritesActiveCoreRulesAndNeverOverwrites</code> (content and non-overwrite). <i>Not automated</i> : the 0700/0600 modes (verified by hand with <code>stat</code> , 2026-09-21).
CLI-8	<i>Not automated</i> . Verified by hand: <code>-help</code> and an unknown flag (2026-09-21).
CLI-9	The launch-URL line's shape is parsed by every end-to-end test run (<code>e2e/lib/server.mjs</code>); <code>TestBackgroundStartsDetachedAndStopStopsIt</code> checks the URL and the background line naming the process. The "serving ... read-only on ..." line is not parsed by any test.
CLI-10	Shape: as CLI-9. Single use: <code>TestLaunchTokenIsSingleUse</code> , <code>TestOnlyURLsUnderThePrefixExist</code> .
CLI-11	The underlying data: <code>TestStatusReportsWeakenedCoreRules</code> , <code>TestCoreDenyCoversKnownSecrets</code> . The startup message text in <code>cmd/fsb</code> is not asserted by a test; verified by hand (2026-09-21).
CLI-12	<code>TestForegroundFsbOnTheUsualPortIsNamedAndStoppable</code> (a foreground <code>fsb</code> stopped with <code>SIGTERM</code> by <code>-stop</code> exits with status 0). The three-second allowance for requests in progress is not tested.
CLI-13	<code>TestRunRejectsABadBrowserNameEarly</code> (an error case); <code>TestBackgroundStartsDetachedAndStopStopsIt</code> (status 0 for <code>-background</code> , <code>-stop</code> ; 1 for <code>-stop</code> with nothing running and for a failed background start, reported once); <code>TestRunShowIgnoreExitsWithoutServing</code> . Status 0 after <code>-help/-init</code> is not asserted; verified by hand (2026-09-21).
CLI-14	<code>TestLoadDeny</code> , <code>TestLoadIgnore</code> , <code>TestParseErrors</code> , <code>TestRuleFileMistakesAreReportedClearly</code> (that loading fails). That <code>cmd/fsb</code> then refuses to start was verified by hand with the real binary (2026-09-21), not by an automated test of <code>run()</code> .
CLI-15	<code>TestChoosePortExplicitIsNeverRemembered</code> , <code>TestRunRejectsAnOutOfRangePortEarly</code> .
CLI-16	<code>TestShowRules</code> (defaults, a file, a missing core rule, the hard-link note), <code>TestRunShowIgnoreExitsWithoutServing</code> .
CLI-17	<code>TestBackgroundStartsDetachedAndStopStopsIt</code> (returns once serving, process keeps running, a second start refused, a failed start reported once and leaves nothing). <i>Not automated</i> : that closing the terminal does not stop it (verified by hand by the user, 2026-10-02).
CLI-18	<code>TestBackgroundStartsDetachedAndStopStopsIt</code> , <code>TestForegroundFsbOnTheUsualPortIsNamedAndStoppable</code> , <code>TestStopNeverSignalsAProcessThatOnlyCallsItselfFsb</code> (a process named <code>fsb</code> by <code>argv[0]</code> , via the PID file and via the remembered port, survives).
CLI-19	<code>TestBackgroundStartsDetachedAndStopStopsIt</code> (background, with address and process), <code>TestForegroundFsbOnTheUsualPortIsNamedAndStoppable</code> (foreground), <code>TestStopNeverSignalsAProcessThatOnlyCallsItselfFsb</code> (an impostor is "not running").
CLI-20	<code>TestForegroundFsbOnTheUsualPortIsNamedAndStoppable</code> (names another <code>fsb</code> and suggests <code>-stop</code>), <code>TestChoosePortAutoFailsLoudlyWhenTheRememberedPortIsTaken</code> .
CLI-21	<code>TestVersionString</code> , <code>TestVersionFlagPrintsAndExits</code> .
ENV-1	<code>TestRulesBuiltFromASymlinkedHomeStillProtectTheRealPath</code> .
CFG-1	<code>TestLoadDeny</code> , <code>TestLoadIgnore</code> (both take one global path; there is no per-folder code path to test against).
CFG-2	By construction: the files are read once, in <code>cmd/fsb</code> 's startup, before any request is served. <i>Not automated</i> : no test starts <code>fsb</code> , edits a rule file, and confirms nothing changes.
CFG-3	<code>TestLoadDeny</code> , <code>TestLoadIgnore</code> (missing-file defaults).
CFG-4	<code>web: prefs.test.ts</code> (fallback on missing or corrupt storage).

Req.	Coverage
CFG-5	TestReadLastPort, TestWriteLastPortThenReadLastPortRoundTrips, TestChoosePortAutoRemembersOnFirstSuccess, TestChoosePortAutoReusesTheRememberedPort, TestChoosePortExplicitIsNeverRemembered.
CFG-6	TestBackgroundStartsDetachedAndStopStopsIt (the PID file names the process and is gone after -stop), TestOwnFilesAreNeverWrittenThroughASymlink. The 0600 mode is set by openOwnFile; not asserted by a test.
NAM-1	TestFoldMatchesTheAliasesAPFSTreatsAsEqual, TestRulesCannotBeDodgedWithFoldingAliases, TestDifferentialEverySpellingOfASecretIsRefused, TestDifferentialEverySpellingOfAnOrdinaryFileIsServed; web: laws.test.ts (filter agreement).
NAM-2	web: format.test.ts (displayName); e2e: “names that would disguise themselves are shown with visible markers”.
ROOT-1	TestInRoots, TestSymlinkEscapingRoot, TestRootContainmentIsByIdentityNotSpelling (<i>skipped unless FSB_CASE_SENSITIVE_VOLUME is set; see Section 13</i>).
ROOT-2	web: goto.test.ts; e2e: “Go to path jumps to any folder inside the served roots, with helpful messages”.
LST-1	TestListStreamsLargeDirectoriesInChunks, TestEmptyDirectoryStillReturnsHeader; e2e: “lists the home folder ...”. Right-alignment of the Size column is a visual property, not asserted by any test.
LST-2	TestListStreamsLargeDirectoriesInChunks; e2e: “lists a 100,000-entry folder quickly with a virtualized list” (<i>opt-in: runs only with FSB_E2E_BIG=1; see Section 13</i>).
LST-3	web: sort.test.ts (including numeric spellings that collate equal, file1 and file01), web: laws.test.ts (total order); e2e: “sorts like eza by default, and Folders first ...”.
LST-4	web: sort.test.ts, web: laws.test.ts; e2e: same as LST-3.
LST-5	TestSymlinkIntoDeniedDirectory, TestListOmitsSymlinkToDeniedTarget, TestListOmitsDeniedHiddenAndEscapingEntries; e2e: “previews each file type correctly and safely” (a link entry). A broken-link row’s on-screen marker is not asserted by an automated test.
LST-6	TestSpecialFilesAreNeverOpened, TestSocketFileIsRefusedLikeAnyOtherSpecialFileNotWithAnInternalError; e2e: “a special file (FIFO) shows a graceful message, not a crash or a stuck Details panel”.
LST-7	web: filter.test.ts; e2e: “lists the home folder without denied or ignored entries” (Hidden files toggle).
LST-8	web: filter.test.ts, web: laws.test.ts (fold agreement with search); e2e: “Match case applies to the filter and to search”.
LST-9	web: format.test.ts (crumbsFor); e2e: “the current-folder link leaves a search and returns to the listing”; copying: e2e “copies the selected path with Alt+C”.
LST-10	web: format.test.ts (pathToHash/parseHash round trip, laws.test.ts).
LST-11	e2e: “resizes a column ...”, “reorders columns with Alt+Arrow”, “reorders columns by dragging a header”, “keeps the layout across a reload, and Reset restores the defaults”, “the Columns menu closes on an outside click and on Escape”; web: columns.test.ts.
LST-12	TestListHidesDeniedAndIgnored, TestEveryEndpointReturns404ForDeniedPaths; e2e: “never lists secret files, even with hidden files shown”.
LST-13	TestRefuseDataless, TestFailMapsDatalessToConflictWithAHelpfulBody (the ErrDataless → 409 mapping every endpoint shares); ListFunc’s own check has no test with a real cloud-only folder (Section 13).
SRC-1	TestSearchEndpoint, TestSearchFindsMatchesBreadthFirst; e2e: “searches by name, shallowest first, without denied or hidden results”.
SRC-2	TestSearchLimitsAndCancellation.

Req.	Coverage
SRC-3	<code>TestSearchFoldsCaseAndUnicode</code> , <code>TestSearchEndpointMatchCase</code> , <code>TestSearchMatchCase</code> , <code>TestSearchMatchCaseStillNormalizesUnicode</code> ; e2e: “Match case applies to the filter and to search”.
SRC-4	<code>TestSearchNeverReportsOrEntersDeniedOrHidden</code> , <code>TestSearchDoesNotFollowSymlinkedDirectories</code> , <code>TestSearchBudgetCountsExcludedEntries</code> (the limit counts entries examined), <code>TestSearchDoesNotDescendIntoAQueuedDirectoryReplacedByASymlink</code> ; the cloud-only case shares LST-13’s citation and the same untested real-folder gap.
SRC-5	e2e: “opens a search result in its folder with the file selected”, “the current-folder link leaves a search and returns to the listing”.
SRC-6	<code>TestSearchReportsFoldersThatCouldNotBeReadCompletely</code> (a read that fails partway), <code>TestSearchReportsAFolderItCouldNotOpen</code> (permission denied on open; a denied folder does not count), <code>TestSearchDoesNotDescendIntoAQueuedDirectoryReplacedByASymlink</code> , <code>TestSearchDoneRecordSaysIncompleteForAFolderThatCannotBeOpened</code> (the done record); <code>web: api.test.ts</code> (“a search reports whether some folders could not be read completely”).
PRV-1	e2e: “toggles the preview pane with a real Space key”, “the preview pane is a checkbox like the other view options”, “the preview pane can be resized ...”, “the preview pane grows with the window instead of stopping at a fixed width”; <code>web: prefs.test.ts</code> (<code>clampPaneWidth</code>).
PRV-2	<code>TestOpenImageAcceptsRasterImagesByContent</code> , <code>TestOpenPDFIdentifiesPDFsByContentOnly</code> , <code>TestListArchiveByContentNotName</code> ; e2e: “shows a real PDF ... , and treats a fake .pdf as text”.
PRV-3	<code>TestHead</code> , <code>TestHeadErrors</code> , <code>TestBinaryContentNeverAppearsInHeadResponses</code> ; e2e: “previews each file type correctly and safely”.
PRV-4	e2e: “previews each file type correctly and safely” (highlighted source; one language exercised end to end). The full language list is not exercised by an automated test beyond <code>web: preview.test.ts</code> ’s language-selection logic.
PRV-5	e2e: “previews each file type correctly and safely” (JSON, CSV); <code>web: preview.test.ts</code> (<code>prettyJSON</code> , <code>parseDelimited</code>).
PRV-6	<code>TestOpenImageAcceptsRasterImagesByContent</code> , <code>TestOpenImageRefusesEverythingElse</code> , <code>TestSniffImage</code> ; e2e: “previews an image through the sandboxed endpoint”.
PRV-7	<code>web: markdown.test.ts</code> ; <code>web: fetchcap.test.ts</code> (the size cap counts bytes as they arrive, whatever the header says; four at a time); e2e: “renders Markdown in a sandboxed frame; raw HTML and scripts stay inert”, “a Markdown file link opens that file inside fsb; a web link opens a new tab”, “Markdown can be viewed as source, and back”.
PRV-8	<code>TestOpenPDFIdentifiesPDFsByContentOnly</code> , <code>TestPDFIsServedInlineOnlyForRealPDFsAndOnlyToOurOwnFrames</code> , <code>TestPDFEndpointObeyDenyRules</code> ; e2e: “shows a real PDF ...” (including focus returning to the list when the viewer takes it). <i>Not automated</i> : a real encrypted PDF (verified by hand by the user, 2026-10-02).
PRV-9	<code>TestListArchiveByContentNotName</code> , <code>TestListArchiveRefusesNonArchives</code> , <code>TestListArchiveNamesAreDefanged</code> , <code>TestListArchiveIsBounded</code> , <code>TestListArchiveStopsAtTheDecompressionLimit</code> , <code>TestTarGzLimitAtAHeaderBoundaryIsIncomplete</code> , <code>TestZipEntryLimitHoldsForValidArchives</code> , <code>TestZipDirectoryByteBudget</code> , <code>TestZipWithAMisleadingCountIsRefused</code> , <code>TestZipWithDataInFrontStillLists</code> , <code>TestZipWrittenToAPipeLists</code> , <code>TestZipModTime</code> , <code>TestArchiveNamesLoseFormatCharacters</code> , <code>TestArchiveNamesAreCutForDisplay</code> , <code>TestPlainTarListingStillSeeksPastMemberBodies</code> (an I/O bound, not only the output), <code>TestTarListingStopsMidStream</code> , <code>TestArchiveListingHonorsItsContext</code> ; <code>FuzzListZipAgreesWithArchiveZip</code> (where both accept an archive); e2e: “lists zip and tar contents without extracting; a fake archive is just text”.

Req.	Coverage
PRV-10	<code>TestIsDataless</code> , <code>TestRefuseDataless</code> (fixture-level; macOS dataless files cannot be created on demand, so this and the endpoints are exercised with a synthetic dataless flag, not a real iCloud or File Provider placeholder; see Section 13).
PRV-11	<code>TestMetaBasics</code> (symlink target); e2e: “previews each file type correctly and safely” (the <code>link-to-main</code> entry).
PRV-12	<code>TestQuickLookIsGivenAPrivateCopyNeverTheUsersPath</code> , <code>TestQuickLookPackageCopyLeavesOutWhatAListingLeavesOut</code> , <code>TestQuickLookFailureIsUnsupported</code> , <code>TestQuickLookDoesNotFollowAFinderAlias</code> (a real alias, real Quick Look), <code>TestThumbnailWithRealQuickLook</code> (including the time limit); <code>TestQuickLookCopyHasATimeLimit</code> , <code>TestQuickLookCopyHonorsACanceledContext</code> , <code>TestCopyStopsWhenItsContextEnds</code> , <code>TestCopyThatExpiredDuringAReadDoesNotSucceed</code> (cooperative cancellation; a blocked read cannot be interrupted, and that is not tested because it cannot be), <code>TestQuickLookRefusesAPackageWithADatalessFolder</code> ; e2e: “shows the Quick Look picture of an Office document, sandboxed like an image” (macOS only). <i>Not automated</i> : real Numbers and Pages files (verified by hand by the user, 2026-10-02) and cloud-only documents (not tried).
HOV-1	e2e: “shows a hover bubble for text files only”.
HOV-2	e2e: same test (cancel and cache are exercised implicitly by repeated hovers in the suite; there is no dedicated timing assertion).
HOV-3	<code>TestHiddenFilesAreStillPeekable</code> ; the core-rule interaction is covered by RUL-13’s tests.
META-1	<code>TestMetaBasics</code> , <code>TestMetaEndpoint</code> .
META-2	<code>TestMetaXattrs</code> , <code>TestEncodeXattr</code> , <code>TestAnEmptyXattrThatGrowsDoesNotPanic</code> , <code>TestAnXattrThatGrowsBetweenTheCallsIsReadAgain</code> , <code>TestAnXattrThatKeepsGrowingIsGivenUpOn</code> , <code>TestAnXattrReadLongerThanItsBufferDoesNotPanic</code> (a seam over the system call makes the race deterministic); e2e: “a binary attribute is labelled hex before its digits”.
META-3	e2e: “fetches the Attributes column lazily, only for rows on screen”.
KEY-1	<code>web: keys.test.ts</code> (<code>keyOf</code>); e2e: “keys on a resize handle only resize: the folder, selection and pane stay”; exercised throughout the end-to-end suite, which drives every shortcut with real key events.
KEY-2	As KEY-1; each key has at least one e2e test naming it (see the citations for <code>/</code> , <code>Alt+S</code> , <code>Alt+G</code> , <code>Alt+C</code> , <code>Space</code> , <code>Escape</code> , <code>Alt+arrow</code>); e2e: “bare “s” now types ahead instead of opening Search (moved to <code>Alt+S</code>)” and its two siblings; “selects, opens and goes up with the keyboard, re-selecting the folder it left”.
KEY-3	e2e: “typing a letter jumps to the entry that starts with it”, “typing the same letter again cycles to the next match”.
MOUSE-1	e2e: “clicking a file selects it instead of downloading it”. <i>Not automated</i> : that a modified click still downloads.
PREF-1	<code>web: prefs.test.ts</code> , <code>web: columns.test.ts</code> ; e2e: “keeps the layout across a reload, and Reset restores the defaults”.
PREF-2	<code>web: prefs.test.ts</code> (malformed and missing storage).
RUL-1	<code>TestOpenDenied</code> , <code>TestEveryEndpointReturns404ForDeniedPaths</code> , <code>TestDeniedIsIndistinguishableFromMissing</code> , <code>TestLawDeniedIsAbsentEverywhere</code> ; e2e: “answers 404, with no content, for denied paths on every endpoint”.
RUL-2	<code>TestHiddenDirectoryIsReachableAndListsItsContents</code> , <code>TestHiddenDirectoryStillReachable</code> .
RUL-3	<code>TestLawAncestorClosure</code> , <code>TestCannotReincludeUnderExcludedDirectory</code> .
RUL-4	<code>TestSymlinkIntoDeniedDirectory</code> , <code>TestSymlinkEscapingRoot</code> , <code>TestListingThroughASymlinkedDirectoryHonoursDenyRules</code> , <code>TestSearchThroughASymlinkedDirectoryNeverReportsDeniedEntries</code> , <code>TestFirmlinkAliasesCannotDodgeDenyRules</code> , <code>TestRootOfSlashHonoursDenyRulesThroughAliases</code> .

Req.	Coverage
RUL-5	<code>TestDifferentialEverySpellingOfASecretIsRefused</code> (thousands of generated spellings, filtered by the operating system), <code>TestDenyRulesSurviveFullCaseFoldingAliases</code> , <code>TestRootOfSlashHonoursDenyRulesThroughFoldedFirmLinkSpellings</code> , <code>FuzzGuardNeverExposesSecrets</code> .
RUL-6	<code>TestHardLinkToADeniedFileIsRefusedWhateverItIsCalled</code> , <code>TestHardLinkCreatedAfterTheIndexWasBuiltIsStillFound</code> , <code>TestFilesWithSeveralNamesAreRefusedUntilTheIndexIsReady</code> , <code>TestLinkIndexWalkIsBounded</code> , <code>TestHardLinkWithANameOutsideEveryLocationIsRefused</code> , <code>TestStaleIndexEntriesAreNotTrusted</code> , <code>TestRenamingAFolderIntoADeniedNameIsSeen</code> , <code>TestAFolderMovedInFromOutsideTheRootsIsSeen</code> , <code>TestAFolderMovedDuringTheWalkIsSeen</code> , <code>TestAFolderReplacedByASymlinkIsSeen</code> , <code>TestARecordedNameNowHoldingAnotherFileIsSeen</code> , <code>TestAnUnreadableFolderLeavesItsNamesUnseen</code> , <code>TestOneNameInTwoSpellingsIsCountedOnce</code> , <code>TestANameSeenTwiceIsCountedOnce</code> , <code>TestNamesChangedDuringTheWalkAreNotTrusted</code> , <code>TestFileWithOneNameDuringTheWalkIsNotExempted</code> , <code>TestNewHardLinkIsRefusedDuringTheRebuildCooldown</code> , <code>TestAPairMadeDuringAWalkIsServedAfterOneMoreWalk</code> , <code>TestARetryRefusedByTheCooldownIsStillOwedAfterIt</code> , <code>TestAFileWithAnUnseenNameDoesNotKeepStartingWalks</code> , <code>TestManyNamesOfOneFileListQuickly</code> , <code>TestAListingNoticesANameAddedWhileItRuns</code> , <code>TestAListingNoticesARenameIntoADeniedNameBetweenBatches</code> , <code>TestASearchNoticesARenameIntoADeniedNameBetweenResults</code> (the one-second memo, tested at its boundary); e2e: “a hard link to a denied file is refused under any name ...”.
RUL-7	<code>TestUnreadableFilesInDeniedDirectoriesLookMissing</code> , <code>TestUnsearchableDeniedDirectoryThroughASymlinkLooksMissing</code> , <code>TestSocketFileIsRefusedLikeAnyOtherSpecialFileNotWithAnInternalError</code> , <code>TestSocketFileAtHTTPLevelAnswersNotFoundNotInternalError</code> .
RUL-8	<code>TestIsDataless</code> , <code>TestRefuseDataless</code> and the per-endpoint dataless checks (see SEC-13); denial is decided in <code>Guard.open</code> before any read, by construction.
RUL-9	<code>TestMatch</code> , <code>TestRegexMetacharsInHomePath</code> , <code>TestParseErrors</code> , <code>TestLawSlashlessPatternsStayWithinOneComponent</code> , <code>TestSlashInsideAClassIsAParseError</code> , <code>FuzzDenyMatchesEverythingUnderADeniedDirectory</code> .
RUL-10	<code>TestMatch</code> , <code>TestMatchBelow</code> .
RUL-11	<code>TestDenyRulesSurviveAwkwardFileNames</code> ; the NAM-1 citations for folding.
RUL-12	<code>TestRuleFilesEditorsProduceWorkOrFailLoudly</code> , <code>TestRuleFileMistakesAreReportedClearly</code> , <code>TestDenyRejectsNegation</code> , <code>TestParseErrors</code> , <code>FuzzDenyFilesRejectNegation</code> .
RUL-13	<code>TestCoreDenyCoversKnownSecrets</code> , <code>TestSecretFilesAreDeniedEverywhereByDefault</code> , <code>TestCredentialFoldersAreDeniedWhereverTheyAre</code> .
RUL-14	<code>TestDenyTemplateActivatesExactlyCore</code> , <code>TestCommonSecretFileNamesAreDeniedAnywhere</code> .
RUL-15	<code>TestCoreDenySecretFilePatterns</code> (documents the costly matches named in the requirement).
RUL-16	<code>TestLoadDeny</code> , <code>TestCoreMissingComparesRulesAsNamesAreCompared</code> , <code>TestStatusReportsWeakenedCoreRules</code> . The startup message text is verified by hand (as CLI-11).
RUL-17	<code>TestIgnoreTemplateParses</code> covers the mechanism; the optional rules themselves are exercised only if a user enables them, which no test does (they are inert by default, by design).

Req.	Coverage
SEC-1	fsb binds 127.0.0.1:0 unconditionally in <code>cmd/fsb</code> ; there is no flag to change it, so there is nothing for a test to try. Exercised implicitly by every test, which all connect to that address.
SEC-2	<code>TestHostValidation</code> , <code>TestDNSRebindingWithValidCookieIsRejected</code> , <code>TestForeignHostRejectedEvenWhenAuthenticated</code> , <code>FuzzMiddlewareOnlyAdmitsLegitimateRequests</code> ; e2e: “a hostile hostname pointing at fsb is refused (DNS rebinding)”.
SEC-3	<code>TestOriginRefererAndFetchMetadata</code> , <code>TestNoCORSHeaders</code> ; e2e: “another site cannot load fsb images ...”, “another site cannot read fsb responses with fetch”, “a navigation started by another site is not authorized by the session”, “a page on another local port (same site) cannot use fsb ...”.
SEC-4	<code>TestOnlyReadMethodsAllowed</code> , <code>TestUnauthenticatedAndReadOnly</code> , <code>TestHandlersDoNotImportFilesystemPackages</code> (no write code exists to call); <code>TestOwnFilesAreNeverWrittenThroughASymlink</code> , <code>TestOwnFilesAreNeverWrittenThroughAHardLinkOrIntoAFIFO</code> (its own files).
SEC-5	<code>TestLaunchTokenIsSingleUse</code> , <code>TestWrongLaunchTokenRejected</code> , <code>TestLaunchTokenOnlyWorksAtRoot</code> , <code>TestExchangeSetsHardenedCookieAndRedirects</code> , <code>TestTokenEntropyAndUniqueness</code> , <code>FuzzLaunchTokenCannotBeReplayedOrMisplaced</code> .
SEC-6	<code>TestExchangeSetsHardenedCookieAndRedirects</code> , <code>TestSessionCookieIsScopedToThePrefix</code> .
SEC-7	<code>TestOnlyURLsUnderThePrefixExist</code> , <code>TestSessionCookieIsScopedToThePrefix</code> ; e2e: “the session cookie is scoped to fsb’s own prefix, so other local servers never see it”, “the session cookie cannot be read by page scripts, and fsb cannot be framed”; <code>web: base.ts</code> covered by <code>laws.test.ts</code> .
SEC-8	<code>TestRequiresSessionCookie</code> , <code>TestAccessLogNeverContainsTheToken</code> .
SEC-9	<code>TestSecurityHeadersOnEveryResponse</code> ; e2e: “produces no script errors or CSP violations”.
SEC-10	<code>TestFileServedSafely</code> .
SEC-11	<code>TestPreviewServesOnlySandboxedImages</code> , <code>TestOpenImageRefusesEverythingElse</code> , <code>TestPDFIsServedInlineOnlyForRealPDFsAndOnlyToOurOwnFrames</code> , <code>TestMarkdownFrameIsServedWithItsOwnStrictPolicy</code> , <code>TestMarkdownFrameRequiresASession</code> , <code>TestQuickLookIsGivenAPrivateCopyNeverTheUsersPath</code> (headers of the picture).
SEC-12	By construction: the binary makes no outbound HTTP call anywhere in its source. Not exercised by a network-monitoring test; verified by code inspection and by the fuzzers never observing one.
SEC-13	<code>TestIsDataless</code> , <code>TestRefuseDataless</code> , <code>TestFailMapsDatalessToConflictWithAHelpfulBody</code> , <code>TestLinkIndexDoesNotEnterDatalessFolders</code> , <code>TestQuickLookRefusesAPackageWithADatalessFolder</code> (the index walk and a package copy), and the per-endpoint refusals they feed (synthetic flag, not a real cloud or File Provider placeholder; see Section 13).
SEC-14	<code>TestDeniedIsIndistinguishableFromMissing</code> , <code>TestNewEndpointsReturn404ForDeniedPathsAndMatchMissingOnes</code> ; e2e: “answers 404, with no content, for denied paths on every endpoint”, “shows the same message for a denied folder and a missing one”. Timing equality is explicitly <i>not</i> claimed or tested (SEC-14 itself says so).

Req.	Coverage
SEC-15	<code>TestQuickLookIsGivenAPrivateCopyNeverTheUsersPath</code> , <code>TestQuickLookCopyCarriesNoExtendedAttributes</code> , <code>TestQuickLookDoesNotFollowAFinderAlias</code> , <code>TestQuickLookPackageCopyLeavesOutWhatAListingLeavesOut</code> , <code>TestThumbnailWithRealQuickLook</code> ; <code>TestCopyStopsWhenItsContextEnds</code> , <code>TestCopyThatExpiredDuringAReadDoesNotSucceed</code> ; <code>FuzzPathParameter</code> , whose stand-in Quick Look answers with the bytes it was given, so a secret reaching it would fail the fuzzer (checked by reverting the guard in the copy).
API-1–API-9	the done lines of API-2 and API-3: the list and search tests of <code>server_test.go</code> (“a listing must end with its done line”), <code>web: api.test.ts</code> (a stream that ends without its done line is an error); <code>server_test.go</code> , <code>m2_test.go</code> and <code>quicklook_test.go</code> exercise every endpoint in the table individually (see the citations above, by feature); <code>FuzzPathParameter</code> (<code>internal/server/fuzz_test.go</code>) fuzzes the <code>path</code> parameter of every endpoint together, and <code>FuzzGuardNeverExposesSecrets</code> (<code>internal/guard</code>) fuzzes the guard beneath them with an oracle independent of fsb’s own rules.
LIM-1	Each limit has a citation under its own feature above (SRC-2, SRC-6, PRV-3, PRV-7, PRV-9, PRV-12, RUL-6, LST-11); <code>web</code> pane-width and column-width clamping in <code>prefs.test.ts/columns.test.ts</code> .
LIM-2	<code>TestSearchLimitsAndCancellation</code> , <code>TestListArchiveIsBounded</code> .
NFR-1–NFR-4, NFR-7	<i>Measured</i> , not continuously tested: the figures in the functional specification are one-time measurements on the machine of Section 11 (2026-09-20/21), not asserted by a test that would fail if they regressed.
NFR-5	<code>go.sum</code> is committed and pinned; a vulnerability scan runs in <code>.github/workflows/ci.yml</code> , which first ran on a hosted runner on 2026-10-02; the commits made since (pinned actions, the pinned scanner version) have not yet run there (see Section 13).
NFR-6	Keyboard operability is exercised throughout the end-to-end suite; semantic markup and <code>prefers-color-scheme</code> are not checked by any automated test (visual/manual only).
FUT-1–FUT-6	Not applicable: none of this is implemented in this version (Section 10.2 of the functional specification).
DST-1, DST-2	Verified by hand at release: the public repository and its license, the version tag, and <code>go install ...@latest</code> producing a working fsb (Section 13).
DST-3	Not applicable: nothing to provide.
DST-4	Partially: the README and this design specification state the security model and the deny-rule costs; the permission-prompt note is written but not exercised by any test (it depends on the user’s own TCC state).
DST-5	Verified by hand: default layout, a custom MANDIR/BINDIR, PREFIX, DESTDIR staging, and <code>uninstall</code> , on the machine of Section 11 (2026-09-21). Not exercised by an automated test.

12.1 Requirements with no automated check

Collected from the table above, these requirements rest on a one-time manual check, on code inspection, or on nothing beyond the absence of a code path: CLI-4 (exact wording), CLI-5 (browser-open failure), CLI-6 (flag wiring), CLI-7 (file modes), CLI-8, CLI-9 (the first output line), CLI-11 (message wording), CLI-12, CLI-13 (success exit codes), CLI-14 (end to end), CFG-2, SEC-12, NFR-1 to NFR-4, NFR-6 (visual half), NFR-7, and DST-4, DST-5. None of these is a security boundary enforced by the guard or the middleware, which is why the heavier testing of Section 8 was spent elsewhere; they are listed here so that claim is checked, not assumed.

13 What Was Not Tested

This is stated up front and in one place, rather than left to be inferred from the absence of a citation in Section 12 or from the environment of Section 11. None of the items below is hidden elsewhere in this document; they are gathered here so a reader does not have to hunt for them.

13.1 Platforms and browsers never run

- **Operating systems other than the one in Section 11.** macOS is the only supported platform (DST-4 is macOS-specific by design), but even within that, only one build and one architecture were tested. Intel Macs, and Apple-silicon Macs other than the model listed, have not run any part of this program. On Linux, the Go tests and the Chrome end-to-end suite run in continuous integration (on GitHub’s hosted runners), but **fsb** has never been used there by hand, and Quick Look previews do not exist there.
- **Firefox.** Not installed on the test machine, not driven by any end-to-end test, and not opened by hand. The interface uses no Chrome-only or Safari-only API knowingly, but this is design intent, not a verified fact for Firefox.
- **Any browser’s mobile mode, and any touch or narrow-window interaction.** The narrow-window column behavior (LST-11) was written to a requirement, not exercised at a phone-sized viewport.
- **A second machine, a second user account, or a network-mounted volume.** Apart from the hosted runners of continuous integration (Section 8), every test ran as one user on one machine. Cross-user permission boundaries (distinct from the deny rules) are provided by the operating system and were not separately exercised.

13.2 Scenarios inside the test suite that did not run this time

- **The 100,000-file scale test** (LST-2) is opt-in (`FSB_E2E_BIG=1`). It runs in continuous integration, on Linux, where the first run (2026-10-02) drew the first rows of 100,000 entries after 118 ms with 35 rows in the page. On macOS it was run by hand on 2026-09-20 (which is what the functional specification’s limits section reports) and again, locally, by the independent reviews of 2026-10-02 (first rows after 80 ms, 36 rows in the page); those are dated measurements on one machine, not a standing result.
- **TestRootContainmentIsByIdentityNotSpelling** (ROOT-1) is skipped unless the environment variable `FSB_CASE_SENSITIVE_VOLUME` names a folder on a case-sensitive volume. It was run once, manually, against a disk image created for that purpose (`hdiutil create -fs "Case-sensitive APFS"`), and is skipped in every other run, including in CI.
- **Real cloud-only (“dataless”) files and folders.** PRV-10, LST-13 and SEC-13 are tested against a synthetic dataless flag set on an ordinary local file or folder (`TestIsDataless`, `TestRefuseDataless` and their callers), because the real flag is set by the File Provider framework itself and cannot be created to order: setting it directly with `chflags` was tried and confirmed not to work (it reports success but the flag does not persist), and a real iCloud Drive or OneDrive placeholder cannot be created in a test fixture or a CI runner. Findings 19 and 20 (Section 10) are exactly this gap: a real File Provider folder was the only thing that ever exercised the code path that was missing. When fixing them, reaching the actual

reported folder to confirm the fix live was also tried and abandoned: the process running the fix could not read `~/Library/CloudStorage` at all (macOS refused it outright), a permission boundary distinct from the one the user’s own session had already been granted, so even this attempt at direct confirmation was itself an instance of the same limitation.

- **A real mouse drag of a column header in Safari.** The Safari driver cannot dispatch a native drag-and-drop sequence, so the browser test (LST-11) falls back to dispatching the underlying drag events directly (logged by the suite as “*safari did not complete a native drop ...; falling back to synthetic drag events*”); this has not been confirmed with an actual pointer drag in Safari.
- **Real cloud-only iWork and Office documents (PRV-12).** The refusal is the same check as every other preview’s and is tested the same synthetic way; no real cloud-only document has been previewed.
- **Encrypted PDFs (PRV-8).** The test moves the focus into the PDF frame itself; a real encrypted PDF, whose viewer takes the focus, was tried only by hand (by the user, 2026-10-02).
- **The clipboard (LST-9, Alt+C).** The end-to-end driver cannot read the system clipboard back in every environment (logged as “*clipboard contents not read back with this driver*”), so the copy action is exercised but the copied text is not always verified.
- **A read blocked in the file system (PRV-12, PRV-9).** Cancellation is cooperative (Section 6.3). The tests use a reader that blocks on command, which shows that expired work fails and that nothing is abandoned; an actual stalled network volume was not tried, and a read it holds up cannot be interrupted by design.
- **Whether the zip reader accepts every archive the standard library does (PRV-9).** The fuzz test compares the two where both accept an archive, and a corpus of archives written by other tools is checked by name; but an implementation that rejected too many valid archives could still pass the fuzz test, so acceptance of the full supported set is not proved.
- **The one-second hard-link verdict (RUL-6).** Tested at its boundary with a short interval and a sleep; the production interval was exercised by an independent review, not by a test in this repository.
- **Search completeness and cloud-only folders (SRC-6).** A cloud-only folder is skipped without marking the result incomplete, by choice, and no real placeholder was searched (see the cloud-only item above).

13.3 Process and infrastructure

- **Continuous integration is new.** It first ran on 2026-10-02, when the repository was given a private remote; that run found three problems (finding 28 of Section 10). It runs the Go tests on macOS and Linux, the front-end tests, fuzzing, `govulncheck`, `npm audit`, and the Chrome suite on Linux, on every push.
- **Safari’s end-to-end suite does not run in CI.** `ci.yml` sets `FSB_E2E_SKIP_SAFARI=1` for the hosted job, because a headless CI runner cannot drive Safari. The Safari suite has been run only on this machine, by hand, each time a feature changed.

- **No independent review has been performed by anyone other than the author and an assistant.** The “independent reviews” recorded in Section 10 and `specs/SECURITY.md` were done by an assistant with access to the source but none to the author’s reasoning, not by a third party, and not by a professional security firm. No penetration test by an independent party was done; the owner decided against one.
- **Installing on another machine.** The release is source only (DST-2), so there is no signing, notarization or Gatekeeper behavior to test, but `go install` from the public repository was tried only once, on the machine of Section 11, right after release.
- **Vulnerability scanning runs on every push, not on a schedule.** `govulncheck` and `npm audit` run in continuous integration; nothing rescans an unchanged commit when a new advisory is published, and no automated dependency updates are proposed.

13.4 What this does not mean

None of the above is a reason to distrust the parts that *were* tested: the guard, the rule matcher, the HTTP middleware and the endpoints are exercised by the heaviest layer of testing this project has (Section 8), including a differential test that asks the operating system itself, not `fsb`, which paths are the same file. The gaps above are concentrated in exactly the places that testing is normally weakest anywhere: process (CI is new), rare environments (a case-sensitive volume, real cloud placeholders, another OS), and interactions a test harness cannot fully simulate (a native drag, a system clipboard). They are listed here so that a reader, in particular someone doing a pre-release review, knows where to look first.